

CAHIER DES CHARGES

Automatisation du Patch Management avec AWX et Ansible

Version pédagogique — programme 3 mois, débutant DevOps

0. Comprendre le projet en 5 minutes

Ce document décrit un projet technique : construire un système qui met à jour automatiquement, en sécurité et de façon tracée, des serveurs Linux et Windows, en s'appuyant sur AWX et Ansible. L'architecture est conçue pour illustrer la capacité d'AWX à orchestrer aussi des équipements réseau et des plateformes de virtualisation — mais ces deux derniers volets restent une vision architecturale : le projet réellement construit et testé se limite à Linux et Windows, pour rester réaliste sur 3 mois avec un débutant DevOps.

Le système repose sur quatre briques qui doivent communiquer entre elles :

- AWX — le cerveau. Il exécute les playbooks Ansible qui appliquent les correctifs.
- Gitea (auto-hébergé) — la bibliothèque. Il stocke et versionne les playbooks Ansible utilisés par AWX.
- Le référentiel de paquets (reprepro) — le dépôt local. Il héberge les correctifs pour ne pas dépendre d'internet à chaque mise à jour.
- Active Directory — le contrôle d'accès. Il gère qui a le droit de faire quoi, en particulier pour les serveurs Windows.

Ce que l'admin doit retenir : le périmètre testé est Linux + Windows. Réseau et virtualisation sont présentés comme la suite logique du projet (mois 4-6 / montée en puissance), pas comme des livrables notés ici.

1. Contexte et objectifs

Un système non mis à jour est une cible de choix pour les cyberattaques et une source d'instabilité, mettant en péril la disponibilité des services et la conformité réglementaire. Ce projet vise à concevoir et déployer une solution de gestion automatisée des correctifs, reposant sur AWX et Ansible, permettant d'orchestrer les mises à jour de manière sécurisée, efficace et traçable.

Objectifs concrets à livrer (périmètre testé)

- Une infrastructure complète installée et configurée sur Proxmox.
- AWX déployé sur un cluster Kubernetes (K3s), intégré à Active Directory.
- Un dépôt Gitea auto-hébergé versionnant l'ensemble des playbooks Ansible.
- Un référentiel de correctifs local géré avec reprepro pour les paquets Debian/Ubuntu.
- Deux workflows d'automatisation fonctionnels et testés : Linux et Windows.
- Une traçabilité complète des exécutions (logs et rapports AWX).

Objectifs de vision (non testés, illustrent la portée d'AWX)

- Automatisation des équipements réseau (Cisco, Juniper) avec sauvegarde/rollback de configuration.
- Orchestration des plateformes de virtualisation (ESXi, OpenStack, Kubernetes, Proxmox) via API.

2. Architecture globale (cible)

L'ensemble de la solution est déployé sur un environnement virtualisé Proxmox à partir du mois 2. Le schéma ci-dessous montre l'architecture cible complète, incluant les éléments de vision (réseau, virtualisation) repérés en pointillé conceptuel — seuls les flux vers les VMs de test Linux et Windows sont effectivement mis en œuvre et testés dans ce projet.

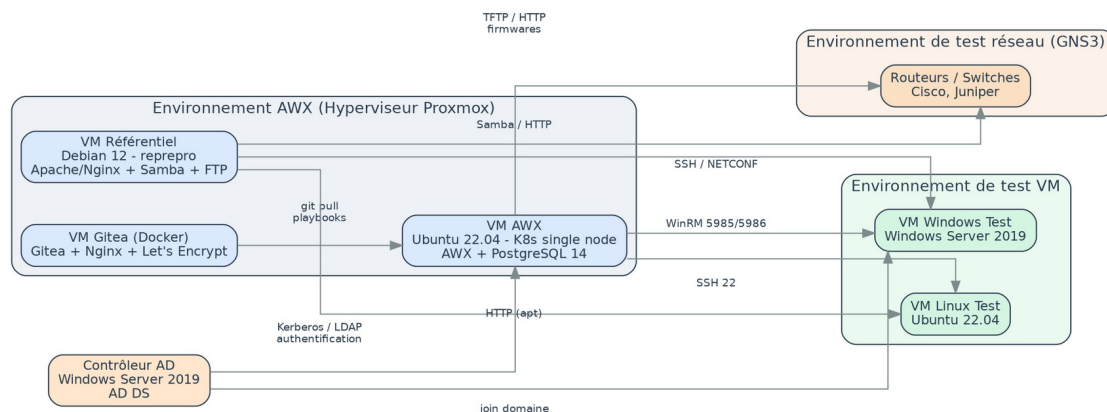


Schéma 1 — Architecture cible de la solution (VMs, flux fonctionnels)

Concrètement : Gitea alimente AWX en playbooks à jour, Active Directory authentifie les accès, et AWX pilote les VMs de test Linux et Windows — directement en SSH/WinRM, en s'appuyant sur le référentiel de paquets pour la distribution des correctifs. Les flux vers GNS3 (réseau) et les API de virtualisation font partie de l'architecture cible mais ne sont pas exercés dans ce projet.

2.1 Flux réseau et protocoles

Chaque flèche du schéma précédent correspond à un protocole réseau précis, utile pour configurer le pare-feu et les règles de sécurité entre VMs.

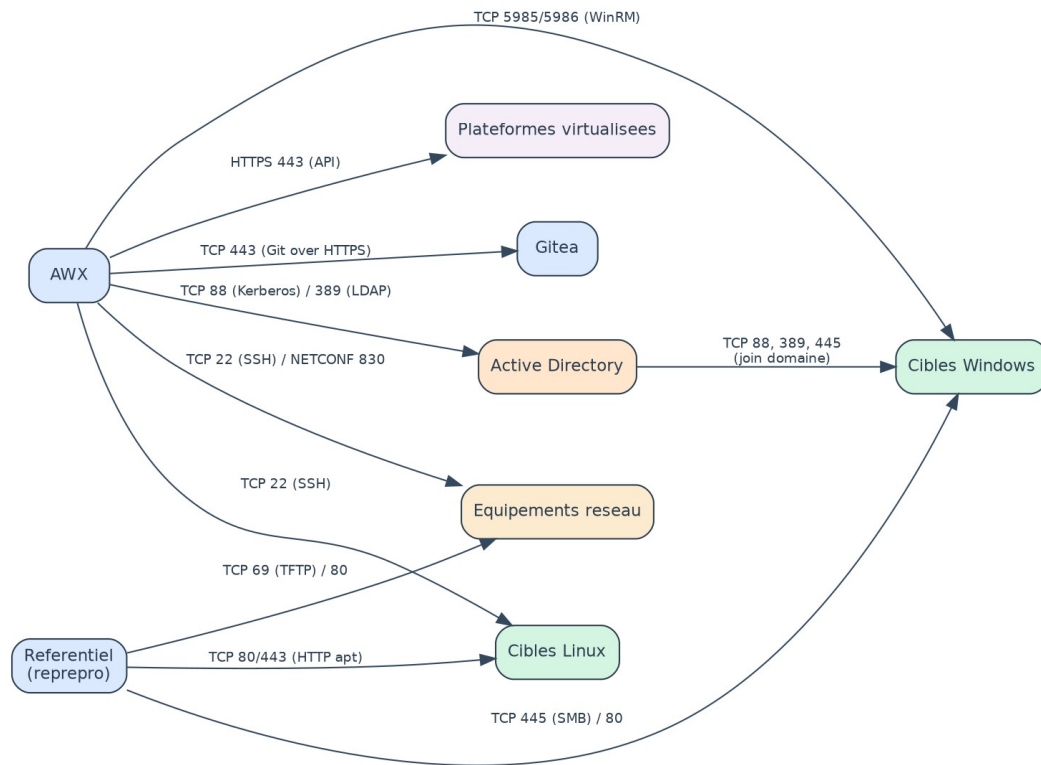


Schéma 2 — Détail des flux réseau (protocoles et ports)

Source	Destination	Protocole / Port	Usage
AWX	Cibles Linux	TCP 22 (SSH)	Exécution des playbooks Ansible
AWX	Cibles Windows	TCP 5985/5986 (WinRM)	Exécution distante Windows
AWX	Active Directory	TCP 88 (Kerberos) / 389 (LDAP)	Authentification centralisée
AWX	Gitea	TCP 443 (Git/HTTPS)	Récupération des playbooks versionnés
Référentiel (reprepro)	Cibles Linux	TCP 80/443 (HTTP apt)	Distribution des paquets .deb
Référentiel	Cibles Windows	TCP 445 (SMB) / 80	Distribution des correctifs Windows
Active Directory	Cibles Windows	TCP 88, 389, 445	Jonction au domaine, authentification
AWX	Équipements réseau / API virtualisation	SSH, NETCONF, HTTPS API	Vision architecturale — non testé

3. Programme pédagogique et livrables par mois

Le projet se déroule en deux temps : un mois d'apprentissage sur un lab jetable (VirtualBox), pour acquérir les bases sans la pression de l'architecture finale, puis deux mois de construction et d'automatisation de l'infrastructure réelle sur Proxmox. Ce découpage permet à l'étudiant de comprendre chaque brique avant de la redéployer « pour de vrai ».

Mois	Semaines	Environnement	Ce qui est construit	Livrable de fin de mois
Mois 1 Fondations	Sem 1-4	Lab jetable (VirtualBox)	Admin Linux + exercice reprepro • Windows Server/AD • Git en local	VM Linux admin. + référentiel reprepro testé • VM Windows/AD • historique Git maîtrisé
Mois 2 Infrastructure	Sem 5-8	Infra cible (Proxmox)	Proxmox + réseau • redéploiement des VMs (Linux, Windows/AD, Référentiel) • Docker + Gitea • K3s	Architecture cible complète en place sur Proxmox, réseau fonctionnel entre toutes les VMs
Mois 3 Automatisation	Sem 9-12	Infra cible (Proxmox)	Ansible (playbooks, inventaires, Vault) • AWX sur K3s connecté à Gitea et au référentiel • Job Templates	AWX exécute avec succès un patching Linux ET Windows de bout en bout, logs et rapports consultables

3.1 Mois 1 — Les Fondations (lab jetable VirtualBox)

Objectif : apprendre les bases sans se soucier encore de l'architecture cible. Tout tourne sur VirtualBox, environnement local et jetable.

- Semaines 1-2 — Administration Linux : shell, permissions, SUDO, APT-GET, SSH, systemctl.
- Exercice pratique intégré : mise en place du référentiel de correctifs avec reprepro sur la VM Linux (structure du dépôt, clé GPG, import de paquets, exposition HTTP). C'est un prolongement naturel du cours APT-GET/systemctl, pas un sujet à part — l'étudiant vérifie qu'un paquet s'installe via apt depuis son propre dépôt.
- Semaine 3 — Windows Server : installation, AD-DS, DHCP, jonction d'un poste au domaine.
- Semaine 4 — Git : philosophie, commandes de base (init, add, commit), branches et fusion, en local d'abord.

Livrable de fin de mois 1 : VM Linux administrée avec un référentiel reprepro fonctionnel et testé, VM Windows/AD opérationnelle, et un historique Git manipulé à la main et compris (pas juste suivi par copier-coller).

3.2 Mois 2 — L'Infrastructure (reconstruction sur Proxmox)

Objectif : construire pour de vrai l'architecture cible du projet. L'étudiant redéploie sur Proxmox ce qu'il a appris en mois 1 — plus rapide cette fois car les concepts sont acquis, et c'est un bon test de compréhension réelle.

- Semaines 5-6 — Proxmox : installation, réseau bridge, stockage LVM, VMs via cloud-init. Redéploiement des VMs Linux, Windows/AD et Référentiel (reprepro) directement sur Proxmox, avec le réseau cible du schéma 1.
- Semaine 7 — Docker : images, volumes, Docker Compose. Déploiement de Gitea (auto-hébergé) sur Proxmox, exposé en HTTPS, premier dépôt créé pour les futurs playbooks.
- Semaine 8 — Kubernetes (K3s) : concepts et installation guidée, préalable technique pour AWX.

Livrable de fin de mois 2 : l'architecture cible complète existe sur Proxmox — VM Linux, VM Windows/AD, VM Référentiel+reprepro, VM Gitea, VM AWX/K3s — toutes provisionnées avec un réseau fonctionnel entre elles.

3.3 Mois 3 — L'Automatisation (le livrable final)

Objectif : faire fonctionner AWX comme chef d'orchestre sur l'infrastructure construite au mois 2.

- Semaines 9-10 — Ansible : inventaires, rôles, playbooks, Ansible Vault.
- Semaine 11 — AWX : déploiement sur K3s, connexion à Gitea (HTTPS/SSH) pour la synchronisation des projets, création des credentials et inventaires.
- Semaine 12 — Projet de patching : playbook Linux (apt update / apt upgrade, en tirant les paquets depuis le référentiel reprepro monté au mois 1) et playbook Windows (win_updates), inventaire dynamique avec groupes Linux/Windows, Job Templates créés et schedulés, tests de bout en bout.

Livrable final testé et démontrable : AWX exécute avec succès un job de patching sur la VM Linux ET sur la VM Windows, avec logs consultables et rapport d'exécution — c'est le critère de réussite central du projet.

4. Détail des machines virtuelles (architecture cible, Proxmox)

Chaque VM a un rôle unique et un dimensionnement adapté à sa charge. Ce dimensionnement est un point de départ à ajuster selon la taille réelle du parc à patcher. Les VMs sont redéployées sur Proxmox au mois 2 (après un premier apprentissage sur VirtualBox au mois 1).

VM / Rôle	OS	CPU	RAM	Stockage	Services clés
AWX (orchestrateur)	Ubuntu 22.04 LTS	8 vCPU	16 Go	100 Go SSD	K3s, AWX Operator, PostgreSQL 14, Ingress
Gitea (dépôt Git)	Ubuntu 22.04 LTS	4 vCPU	8 Go	200 Go SSD	Docker, Gitea, MariaDB/PostgreSQL, Nginx + Let's Encrypt
Référentiel de correctifs	Debian 12	4 vCPU	16 Go	2 To HDD	reprepro, Apache/Nginx, Samba, FTP
Contrôleur Active Directory	Windows Server 2019	4 vCPU	8 Go	100 Go SSD	AD DS, Kerberos/WinRM
Linux Test	Ubuntu 22.04	2 vCPU	4 Go	40 Go SSD	Cible testée du patching Linux
Windows Test	Windows Server 2019	2 vCPU	4 Go	60 Go SSD	Cible testée du patching Windows
Équipements réseau (optionnel)	GNS3	-	-	-	Illustratif — non testé dans ce projet

5. Le référentiel de correctifs avec reprepro

Plutôt qu'un mirroring générique, le référentiel Linux/Debian repose sur reprepro, l'outil de référence pour gérer un dépôt APT local signé. Il permet de contrôler précisément quels paquets et quelles versions sont disponibles pour les machines du parc, avec une signature GPG garantissant l'intégrité des paquets distribués.

Dans ce projet, reprepro est introduit dès le mois 1 comme exercice pratique du cours d'administration Linux (semaines 1-2) sur le lab VirtualBox, puis redéployé à l'identique sur la VM Référentiel de l'architecture Proxmox au mois 2. Au mois 3, c'est ce même dépôt que le playbook Ansible de patching Linux interroge.

Principe de fonctionnement

- Un dépôt reprepro est structuré par distribution (ex : jammy, bookworm) et par composant (main, security...).
- Les paquets sont importés avec reprepro includedeb <distribution> <fichier.deb> ou synchronisés depuis les miroirs officiels avec reprepro update.
- Le dépôt est signé avec une clé GPG dédiée ; les clients apt doivent faire confiance à cette clé.
- Le dépôt est exposé en HTTP/HTTPS via Apache ou Nginx pour être consommé par apt sur les machines cibles.
- Pour Windows, la même VM référentiel peut héberger en parallèle les binaires de mise à jour, distribués via Samba ou HTTP.

Point d'attention pour l'admin : reprepro ne mélange pas les paquets « validés en test » et « en attente de validation » dans la même distribution — utiliser deux distributions (ex. jammy-staging et jammy-prod) permet un vrai contrôle qualité avant la mise en production d'un correctif.

6. Workflows d'automatisation

Deux workflows sont construits et testés de bout en bout dans ce projet (Linux, Windows). Deux autres illustrent la portée qu'AWX pourrait avoir sur un parc plus large (réseau, virtualisation), sans être mis en œuvre ici.

6.1 Serveurs Linux

✓ TESTÉ ET VALIDÉ DANS LE PROJET

Mise à jour via les gestionnaires de paquets natifs (apt), en tirant les paquets depuis le référentiel reprepro. Détection, vérification des dépendances, application contrôlée et redémarrage planifiable.

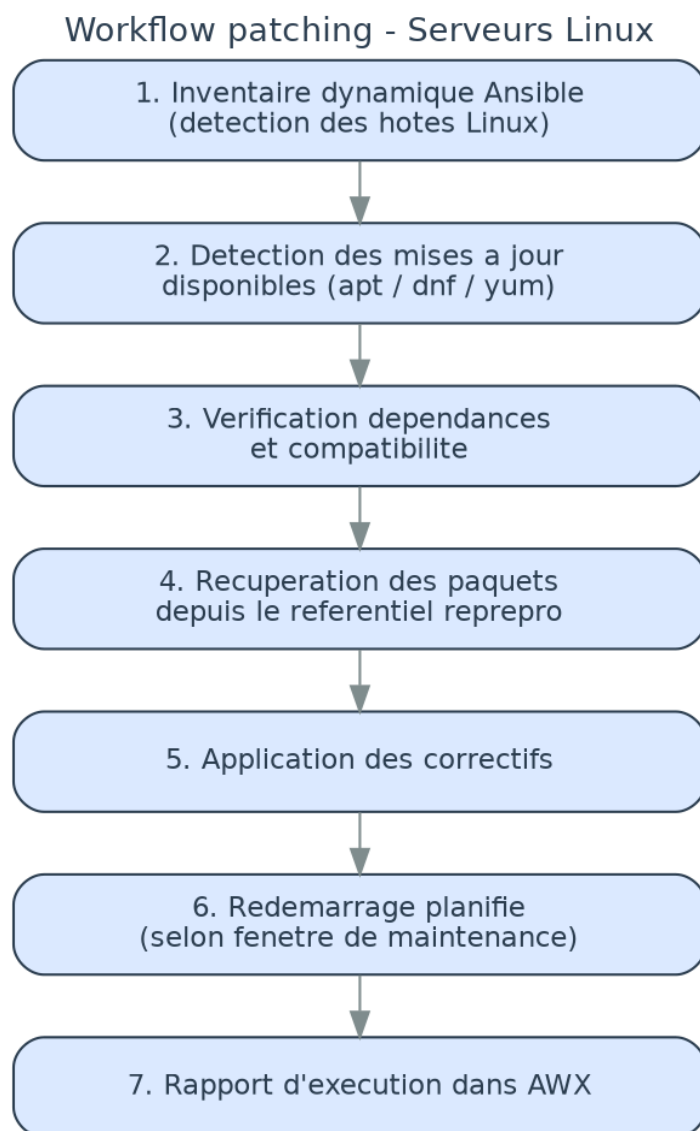


Schéma 3 — Workflow de patching Linux (testé dans ce projet)

6.2 Serveurs Windows

✓ TESTÉ ET VALIDÉ DANS LE PROJET

Exécution distante sécurisée via WinRM après authentification Active Directory ; les correctifs sont appliqués via le module win_updates, avec reporting des erreurs.

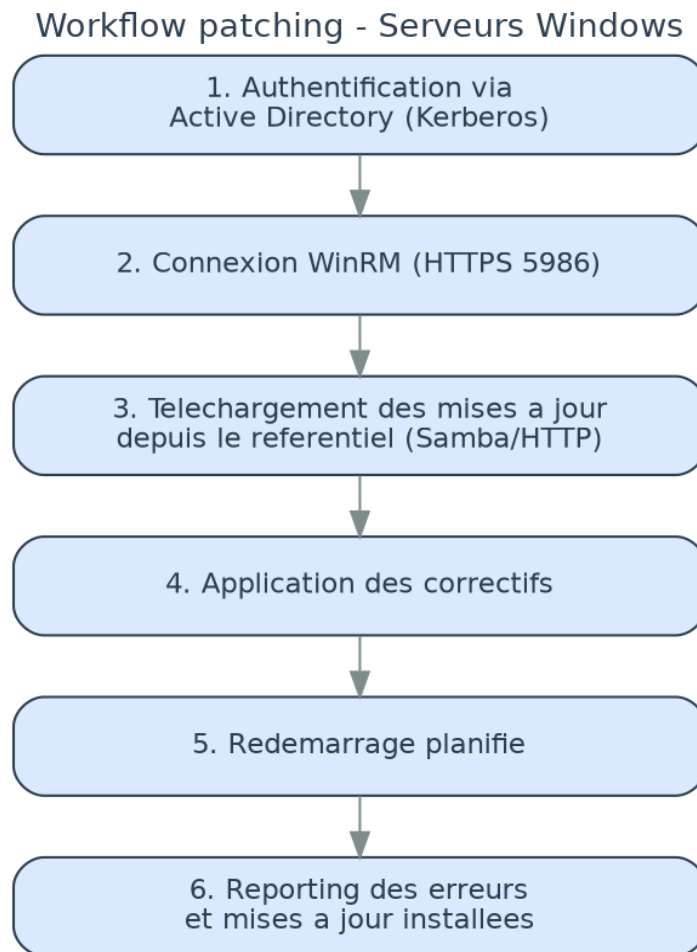


Schéma 4 — Workflow de patching Windows (testé dans ce projet)

6.3 Équipements réseau (Cisco, Juniper...)

🕒 VISION ARCHITECTURALE — NON TESTÉ DANS CE PROJET

Approche envisagée : sauvegarde systématique de la configuration avant toute action (point de rollback), puis application de la nouvelle image, suivie d'une vérification post-déploiement. Les modules Ansible réseau (`ios_facts`, `ios_command`) sont abordés en formation Ansible, mais ce workflow complet n'est pas construit ni testé dans ce projet.

Workflow patching - Equipements reseau

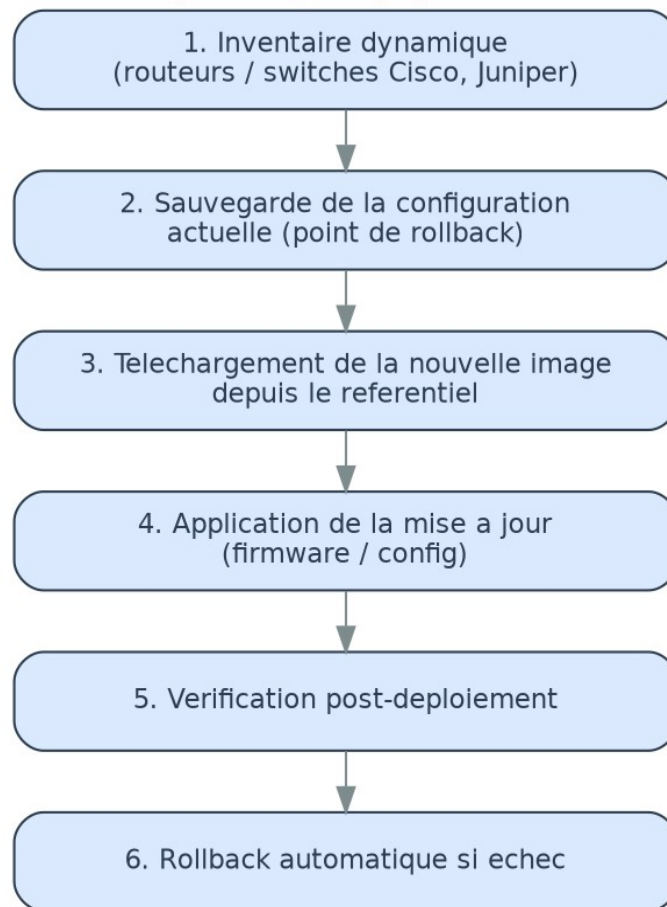


Schéma 5 — Workflow de patching réseau (vision architecturale)

6.4 Plateformes virtualisées (ESXi, OpenStack, Kubernetes, Proxmox)

🕒 VISION ARCHITECTURALE — NON TESTÉ DANS CE PROJET

Approche envisagée : orchestration non intrusive via les API de chaque plateforme, mode maintenance avant application du correctif, puis test de santé post-mise à jour. Ce volet correspond à une montée en compétence ultérieure (au-delà des 3 premiers mois) et n'est pas testé dans ce projet.

Workflow patching - Plateformes virtualisees (ESXi / OpenStack / K8s / Proxmox)

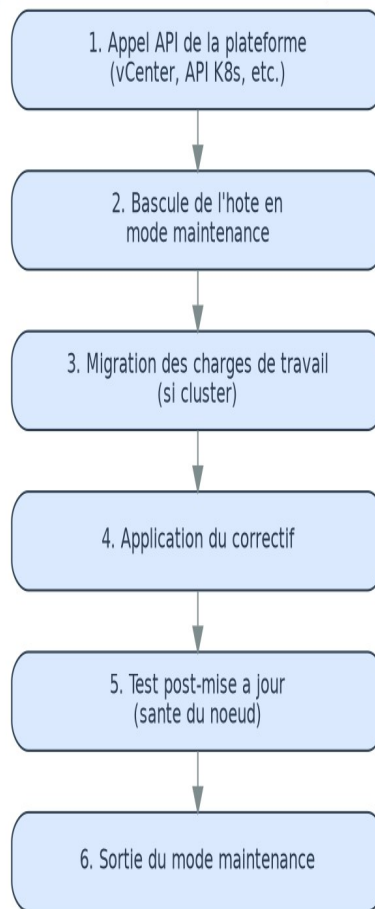


Schéma 6 — Workflow de patching virtualisation (vision architecturale)

7. Sécurité et contrôle d'accès

- Authentification centralisée via Active Directory pour tous les accès administratifs.
- Rôles et permissions granulaires dans AWX (principe du moindre privilège).
- Connexions SSH et WinRM sécurisées par une gestion rigoureuse des clés et certificats.
- HTTPS/TLS pour l'accès web à AWX et Gitea (reverse proxy Nginx + Let's Encrypt).
- Signature GPG des paquets du référentiel reprepro.
- Sauvegarde automatique des configurations (VMs) pour une récupération rapide en cas d'incident.

8. Critères de réussite du projet

Le projet est considéré comme abouti lorsque l'infrastructure suivante est opérationnelle et démontrable :

- Proxmox héberge l'ensemble des VMs du projet, correctement dimensionnées et réseautées (mois 2).
- AWX est accessible en HTTPS, intégré à Active Directory, et exécute des playbooks synchronisés depuis Gitea.
- Le référentiel reprepro sert des paquets signés à la VM Linux de test avec succès.
- Les workflows Linux et Windows ont été exécutés au moins une fois de bout en bout, avec rapport d'exécution consultable dans AWX.

- L'étudiant peut réexpliquer chaque étape (Git, reprepro, Ansible, AWX) sans notes — critère qualitatif central pour un profil débutant.

Les workflows réseau et virtualisation (schémas 5 et 6) restent présentés dans ce document comme perspective d'évolution du projet, à ne pas exiger comme critère de validation des 3 premiers mois.

9. Liens utiles pour la réussite du projet

Proxmox

- [Documentation officielle Proxmox VE](#)
- [Proxmox — réseau et bridges](#)

Git

- [Documentation officielle Git](#)
- [Pro Git \(livre gratuit en français\)](#)

Ansible

- [Documentation Ansible](#)
- [Ansible — modules Windows \(WinRM\)](#)
- [Ansible — inventaire dynamique](#)
- [Ansible — collection Cisco IOS \(vision réseau\)](#)
- [Ansible — collection VMware \(vision virtualisation\)](#)

AWX

- [Documentation officielle AWX](#)
- [AWX Operator \(déploiement Kubernetes/K3s\)](#)
- [Dépôt GitHub AWX](#)

Kubernetes / K3s

- [Documentation officielle Kubernetes](#)
- [K3s — documentation officielle](#)

Gitea et Docker

- [Documentation officielle Gitea](#)
- [Gitea avec Docker Compose](#)
- [Documentation Docker](#)

Référentiel de paquets (reprepro)

- [reprepro — documentation officielle \(Debian wiki\)](#)
- [reprepro — page manuel](#)
- [reprepro — dépôt source](#)

Active Directory / Windows Server

- [Documentation Active Directory Domain Services](#)
- [Configuration de WinRM pour Ansible](#)

Reverse proxy et TLS

- [Let's Encrypt / Certbot](#)

Pour aller plus loin (vision réseau / virtualisation)

- [GNS3 — documentation](#)
- [Ansible NETCONF](#)